Paper seminar

Week 4.5: A Taxonomy of Policy Gradients methods

A brief overview of the algorithms behind REINFORCE, PPO, DAPO, A*PO and beyond

Carrie Chen and CUAI team

REINFORCE

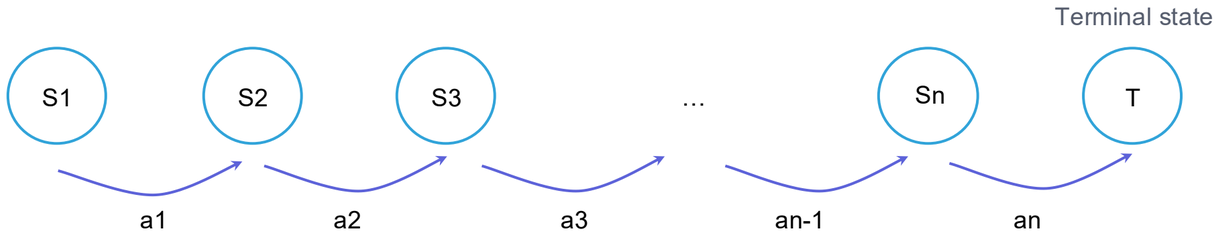
Recap Markov Decision Processes

let $\tau = (s_1, a_1, \ldots, s_T, a_T)$ *be a trajectory*

Finite horizon setting

Objective:

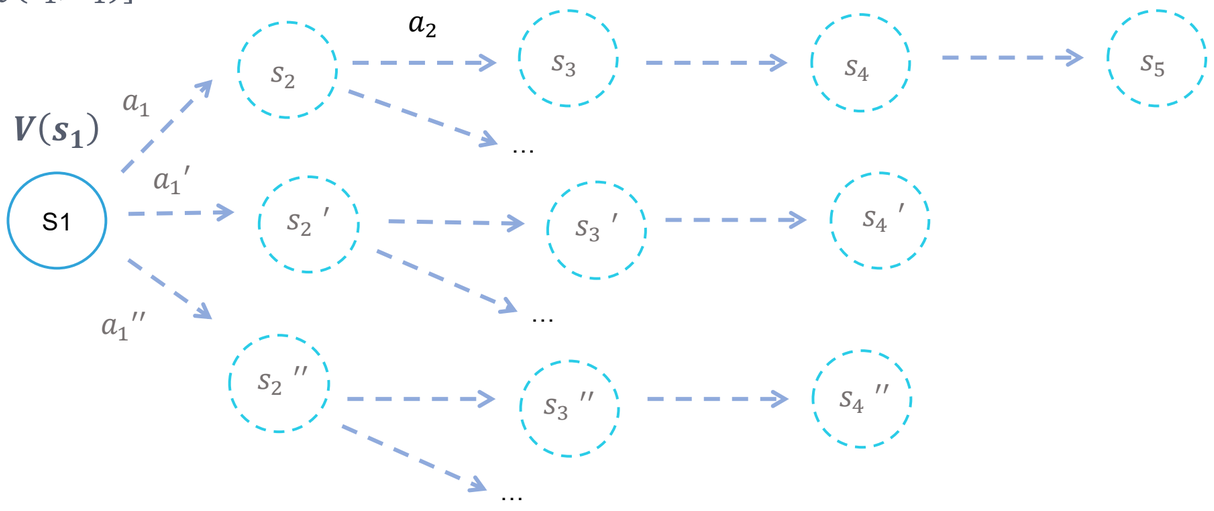
$$J(\theta) = \max_{\theta} E\left[\sum_{n=1}^{N} \gamma^{n-1} r(s_n, a_n)\right]$$



V-function, expected reward at state: $V(s) = E_{a\sim\pi(\cdot|s)}[r(s,a) + \gamma r(s',a') + \gamma^2 \ldots]$

$$V(s_1) = E_{a_1\sim\pi(\cdot|s_1)}[r(s_1,a_1) + \gamma r(s_2,a_2) + \gamma^2 \ldots]$$

$$= E_{a_1\sim\pi(\cdot|s_1)}[Q(s_1,a_1)]$$



Optimal policy: action that maximize the discounted expected reward $\pi^*(s_1) = \max_a r(s_1,a) + \gamma E_{s_2}[V(s_2)]$

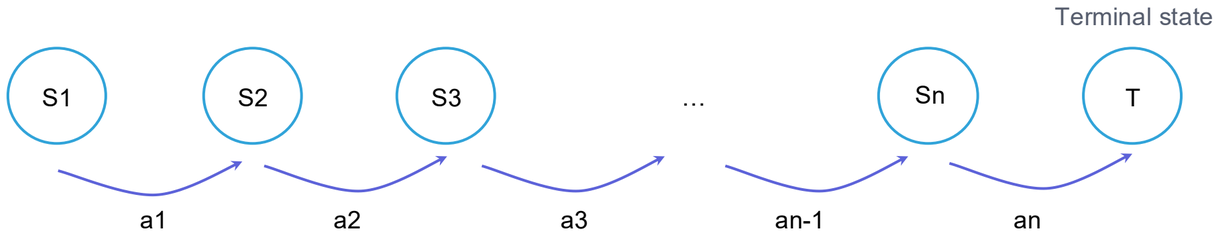
Recap Markov Decision Processes

let $\tau = (s_1, a_1, \ldots, s_T, a_T)$ *be a trajectory*

Finite horizon setting

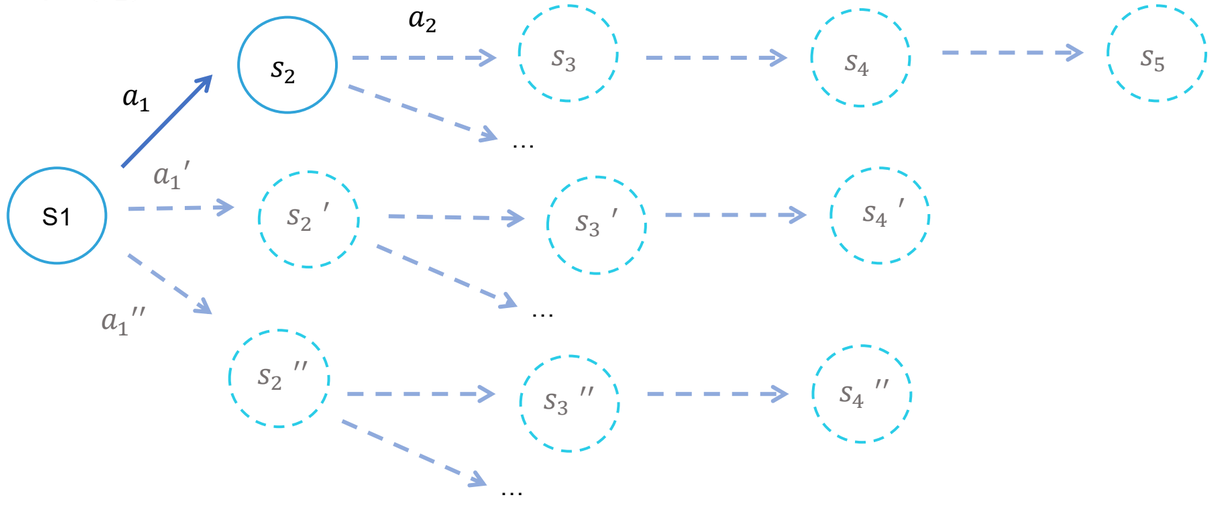
Objective:

$$J(\theta) = \max_{\theta} E\left[\sum_{n=1}^{N} \gamma^{n-1} r(s_n, a_n)\right]$$



Q-function, expected reward at state s after taking action a: $Q(s,a) = r(s,a) + E_{s' \sim P(\cdot|s,a)}[\gamma r(s',a) + \gamma^2 \ldots]$

$$Q(s_1, a_1) = r(s_1, a_1) + \gamma E_{a_2 \sim \pi(\cdot|s_2)}[r(s_2, a_2) + \gamma \ldots]$$
$$= r(s_1, a_1) + \gamma V(s_2)$$



Optimal policy: action that maximize the discounted expected reward $\pi^*(s_1) = \max_a Q(s_1, a)$

Recap Deep Q-Learning

Optimal policy: selects action that maximize the discounted expected reward $\pi^*(s_1) = \max_a Q(s_1, a)$

→ If we have a good Q function, the optimal action at each state can be computed by being greedy w.r.t. Q

→ How do we learn the Q function? Supervised learning, let θ parameterize Q!

Train the Q-network to minimize the MSE:

$$L(\theta) = (y - y_{pred})^2$$

$$= E_{(s,a,s',r)\sim\rho}[\left(y - Q^\theta(s,a)\right)]^2$$

$$= E_{(s,a,s',r)\sim\rho}[\left((r(s,a) + \gamma \max_{a'} Q^{\theta^-}(s,a)) - Q^\theta(s,a)\right)]^2$$

Use a mini-batch size |B| of $B = \{(s_i, a_i, r_i, s_i')\}_{i=1}^{|B|}$ tuples to estimate the expectation:

$$L(\theta) = \sum_{(s,a,s',r)} P_\rho\left((s,a,s',r)\right)[\left((r(s,a) + \gamma \max_{a'} Q(s',a)) - Q^\theta(s,a)\right)]^2$$

Probability of sampling tuple (s,a,s',r) under ρ

$$\approx \frac{1}{|B|}\sum_{(s,a,r,s')\sim B}\left[Q(s,a) - r(s,a) + Q^\theta(s,a)\right]^2$$

here, ρ is the batch so if we uniformly sample each tuple $P_\rho\left((s,a,s',r)\right) = \frac{1}{|B|}$

$$\theta \leftarrow \theta - \eta \frac{1}{|B|}\sum_{(s,a,r,s')\sim B}\left[Q(s,a) - r(s,a) + Q^\theta(s,a)\right]^2 \nabla_\theta Q^\theta$$

Mnih, 2013 (deepmind)

Deep NN to learn a Q function that can successfully play Atari games!

Estimator

REINFORCE and importance weighting

Q-learning is only a surrogate process (through the action space) for the ultimate objective: maximizing expected total discounted reward over an entire trajectory

→ Even if you learn the optimal Q function, you still need to argmax over it to find the best action at each state

$$\pi^*(s) = \max_a Q(s,a)$$

→ At **each** state, run SGD on Q to find it's maximum… slow since Q^θ is an arbitrary, nonlinear function (NN)

→ What if we parameterized and learned π^* directly? This means we can sample $\hat{y} = a^* = \pi^\theta(s_1)$

Examples of π^θ parameterizations in the discrete action space setting:

Softmax policy

$$\theta_{s,a} \in \mathbb{R}, \forall s, a \in S \times A$$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta_{s,a})}{\sum_{a'} \exp(\theta_{s,a'})}$$

Softmax linear policy

Feature vector $\phi(s,a) \in \mathbb{R}^d$, and parameter $\theta \in \mathbb{R}^d$

$$\pi_\theta(a \mid s) = \frac{\exp(\theta^\top \phi(s,a))}{\sum_{a'} \exp(\theta^\top \phi(s,a'))}$$

Neural policy

Neural network $f_\theta : S \times A \mapsto \mathbb{R}$

$$\pi_\theta(a \mid s) = \frac{\exp(f_\theta(s,a))}{\sum_{a'} \exp(f_\theta(s,a'))}$$

Think of π^θ as a classifier over the entire action space $\Delta(A)$ that takes a state as its input and outputs the optimal a^*

Williams, 1992

The likelihood-ratio identity. Increase the probability of actions appearing in trajectories with high reinforcement, decrease it for low reinforcement

Estimator

REINFORCE and importance weighting

Q-learning is only a surrogate process (through the action space) for the ultimate objective: maximizing expected total discounted reward over an entire trajectory

Let $J(\theta)$ measure the expected reward of π^θ over an entire trajectory, our objective is

$$\max_\theta J(\theta) = \max_\theta E_{\tau\sim\pi^\theta}\,[R(\tau)]$$

$$\nabla_\theta J(\theta) = \nabla_\theta\, E_{\tau\sim\pi^\theta}\,[R(\tau)]$$

$$= \nabla_\theta \sum_\tau P_{\pi_\theta}\,(\tau)\,R(\tau)$$

Clearly, summing over all possible trajectories is intractable so let's construct an unbiased estimate of it from a mini-batch of trajectories sampled via some frozen iteration of π^θ denoted $\pi^{\theta 0}$: $B = \{(\tau_i, r(\tau_i)\}_{i=1}^{|B|}$

$$\nabla_{\theta 0} J(\theta^0) \approx \frac{1}{|B|}\nabla_{\theta 0} \sum_{i=1}^{|B|} P_{\pi^{\theta 0}}(\tau_i)\,R(\tau_i)$$

For the first iteration of SGD: $\theta^1 \leftarrow \theta^0 - \eta\nabla_{\theta 0} J(\theta^0)$, our gradient estimate is on policy but for θ^2:

$$\theta^2 \leftarrow \theta^1 - \eta\nabla_{\theta^1} E_{\tau\sim\pi^{\theta^1}}\,[R(\tau)]$$

what we need

$$\theta^2 \leftarrow \theta^1 - \eta\nabla_{\theta^1} E_{\tau\sim\pi^{\theta^0}}\,[R(\tau)]$$

what we have :(

Our old batch of trajectories is no longer on policy…

→ 1. Reconstruct B under θ^{n-1} for each update (expensive)

→ 2. Reweight B constructed from θ^0 under θ^{n-1} for each update (free) (importance weighting!!)

Williams, 1992

The likelihood-ratio identity. Increase the probability of actions appearing in trajectories with high reinforcement, decrease it for low reinforcement

Estimator

A recap of Importance Weighting

How do we re-use trajectories sampled from an earlier (old) iteration of our policy to update the current version without introducing bias/staleness???

Recall that the first iteration of SGD: $\theta^1 \leftarrow \theta^0 - \eta \nabla_{\theta 0} J(\theta^0)$ works, but for θ^2:

$$\theta^2 \leftarrow \theta^1 - \eta \nabla_{\theta^1} \underbrace{E_{\tau \sim \pi^{\theta^1}}}_{\text{what we need}} [R(\tau)] \qquad \theta^2 \leftarrow \theta^1 - \eta \nabla_{\theta^1} \underbrace{E_{\tau \sim \pi^{\theta^0}}}_{\text{what we have :(}} [R(\tau)]$$

Let π^{θ^0} be our sampling distribution such that $\mathrm{P}\left(\frac{P\pi^{\theta^1}(y)}{P\pi^{\theta^0}(y)}\right) < \infty, \forall y$

$$\begin{aligned} E_{\tau \sim \pi^{\theta^1}}[R(\tau)] &= \sum_\tau P\pi^{\theta^1}(\tau) R(\tau) \\ &= \sum_\tau \frac{P\pi^{\theta^0}(\tau)}{P\pi^{\theta^0}(\tau)} \pi^{\theta^1}(\tau) R(\tau) \\ &= \sum_\tau \pi^{\theta^0}(\tau) \frac{P\pi^{\theta^1}(\tau)}{P\pi^{\theta^0}(\tau)} R(\tau) \\ &= E_{\tau \sim \pi^{\theta^0}} \left[\frac{P\pi^{\theta^1}(\tau)}{P\pi^{\theta^0}(\tau)} R(\tau) \right] \end{aligned}$$

Let $\rho = \frac{P\pi^{\theta^1}(\tau)}{P\pi^{\theta^0}(\tau)}$, using the log trick $\nabla \pi^\theta(\tau) = \pi^\theta(\tau) \nabla \log(\pi^\theta(\tau))$ we can write the final gradient as:

$$E_{\tau \sim \pi^{\theta^0}} \left[\rho R(\tau) \sum_{|\tau|} \nabla_{\pi^{\theta^1}} \log(\pi^{\theta^1}(a_t | s_t)) \right]$$

Williams, 1992

The likelihood-ratio identity. Increase the probability of actions appearing in trajectories with high reinforcement, decrease it for low reinforcement

Estimator

All subsequent algorithms modify the following

$$w_t \cdot R(y)_t \cdot \nabla \log \pi_\theta(a_t \mid s_t)$$

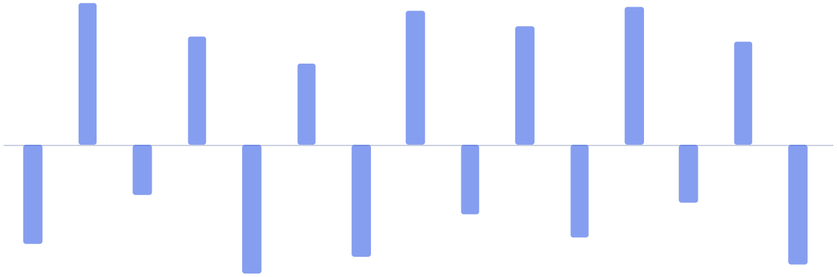
IS / trust region	credit / baseline	policy gradient
How closely should policy updates be constrained together between iterations?	How do we assign and measure credit (reward) to each action/token across trajectories?	Which policy probabilities should receive gradient, and in what direction?

Almost every modern algorithm changes one of these three axes

Advantage: Same gradient in expectation, far less noise

REINFORCE has high variance, we should replace raw returns with relative ones so actions receives positive gradients only when they perform significantly better than the average expected return from that state

Returns (raw) R_t



advantages A_t

Let π^{θ^0} be our sampling distribution such that $\mathrm{P}\left(\frac{P\pi^{\theta^1}(y)}{P\pi^{\theta^0}(y)}\right) < \infty, \forall y$

Unbiased

the E[policy gradient] is unchanged

›

Lower variance

the spread collapses around zero

›

But

now there is another model to learn

TRPO: we should constrain the mag. of gradient steps

Small changes in policy's parameters space can lead to large changes in policy behavior so is there a way to optimize the parameters while staying cognizant of changes in the policy itself??

→ implicitly, PG considers Euclidian distances in parameter space but we actually care about information changes in policy space…

→ How do we measure "policy/behavioral" changes? Measure changes in the distribution itself!

At iteration t, our objective is as follows where $\rho_t = \frac{P\pi^{\theta^t}(\tau)}{P\pi^{\theta^0}(\tau)}$ and π^{θ_0} is your sampling distribution:

$$\max_{\theta} E_{\tau\sim\pi^{\theta_0}}[\rho_t \cdot \hat{A}_t]$$

subject to

$$E_{\tau\sim\pi^{\theta_0}}\left[\frac{1}{|\tau|}\sum_{i=1}^{|\tau|} D_{KL}\left(\pi^{\theta_0}(a_t|s_t) \,||\, \pi^{\theta_t}(a_t|s_t)\right)\right] < \delta$$

Token level KL

But how do we compute the KL distance between trajectories?

→ Tdlr: this is really expensive because it requires inverting the hessian during taylor expansion of D_{KL}, $O(n^3)$

$$D_{KL}\left(P\pi^{\theta_0}(\tau) \,||P\pi^{\theta_t}(\tau)\right) = E_{\tau\sim\pi^{\theta_0}} ln\frac{P\pi^{\theta^t}(\tau)}{P\pi^{\theta^0}(\tau)} = E_{\tau\sim\pi^{\theta_0}} \frac{1}{|\tau|}\sum_{i=1}^{|\tau|} ln\frac{\pi^{\theta^t}(a_t|s_t)}{\pi^{\theta^0}(a_t|s_t)}$$

Cannot solve closed form… uses taylor expansion to approximate

Failure

destructive policy drift

Modification

an explicit KL trust region

Cost

constrained optimization is cumbersome

Trunk

PPO: put the trust region inside the loss

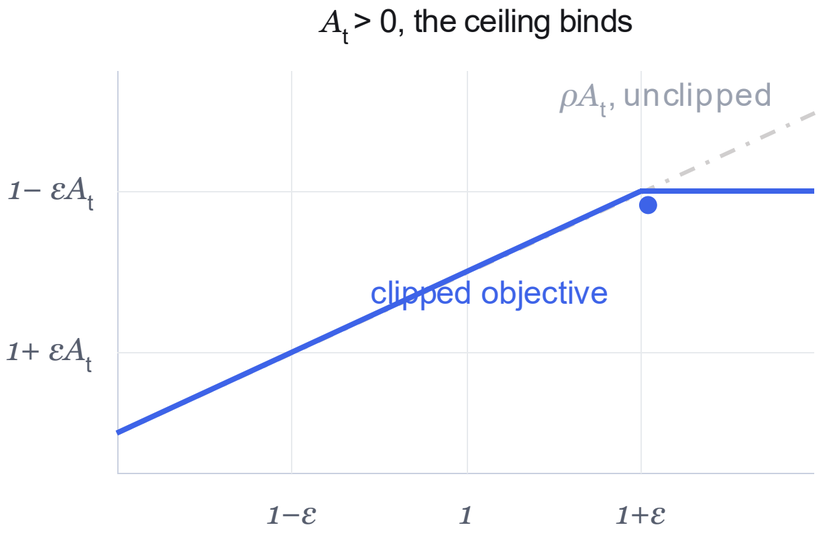
TRPO achieved SOTA performance against existing methods i.e. PG, REINFORCE, SAC at the time but was too slow to scale/execute on low latency tasks like Atari games.. Is there a better way?

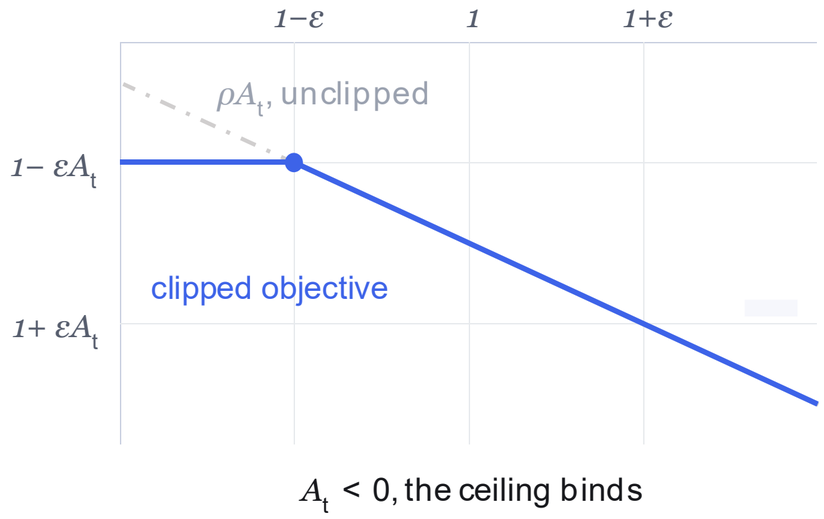
- → Construct a mini-batch Supervised Learning style objective using $B = \{(s_i, a_i, A_i, s_i')\}_{i=1}^{|B|}$
- → Use a heuristic clipping function to force ”closeness” to the base policy instead of using the KL distance

At iteration t, our objective is as follows where $\rho_t = \frac{P\pi^{\theta^t}(\tau)}{P\pi^{\theta^0}(\tau)}$ and π^{θ_0} is your sampling distribution:

$$\min_{\theta} \frac{1}{|B|} \sum_{i=1}^{|B|} min\left\{\rho_t \cdot \hat{A}^{\pi^{\theta_t}}(a_t, s_t), clip(\rho_t, 1-\varepsilon, 1+\varepsilon) \cdot \hat{A}^{\pi^{\theta_t}}(a_t, s_t)\right\}$$

Visualizing how clipping works on both polarities of A:





The payoff: multiple minibatch epochs over the same rollouts (hardware friendly!), without the constrained solve

Fork

Three design decisions to the same reference algorithm

Modern policy-gradient methods branch from PPO by choosing whether to remove the critic, repair it, or avoid estimating the current-policy advantage altogether

REINFORCE
$R \nabla \log \pi$

› reduce variance

AC/GAE

› bound policy drift

TRPO
a hard KL constraint

› make it SGD-friendly

PPO
ratio clipping, minibatch reuse

Proximal Policy Optimization

"the critic is too expensive, throw it away"

Critic-free

GRPO · RLOO · REINFORCE++ · Dr. GRPO · DAPO · CISPO · GSPO SAPO · ASPO · STAPO · GiGPO

"the critic is not the fundamental problem, fix it"

Fix the critic

VAPO · Direct-Advantage DAPO

"do not estimate the current advantage at all"

Sidestep it

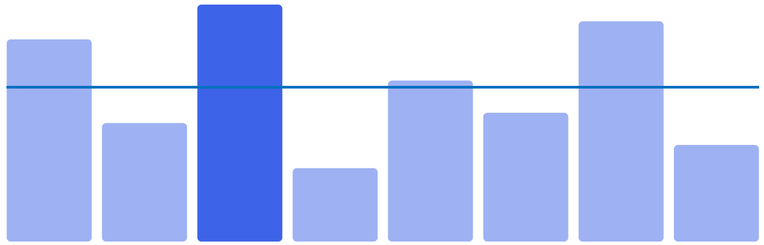
A*-PO

Full taxonomy tree with paper citations available as a demo on our github :D

Patches

GRPO: the group is the baseline

Instead of learning a critic, can we use test time scaling to estimate relative advantage for a single trajectory directly by sampling multiple responses to the same prompt?



group mean, the baseline

$$\hat{A}_i = \frac{R_i - \text{mean}(R_1, \ldots, R_G)}{\text{std}(R_1, \ldots, R_G)}$$

What changed

$A_{t\,GAE} = G_t - V_\varphi(s_t) \rightarrow A_{i\,GRPO}$

Price paid

1 critic $\rightarrow$ G generations / prompt

And token-level temporal credit assignment is gone. That gap spawns the next six papers…

Patches

RLOO: leave one out

Use the other sampled responses as the baseline, avoiding both a learned critic and the bias from including a sample in its own baseline

i

the other G − 1 responses, averaged

$$\hat{A}_i = R_i - \frac{1}{G-1} \sum_{j \neq i} R_j$$

PPO

the full actor-critic stack

› “do we need the critic?”

asked directly

› RLOO and GRPO

a return toward REINFORCE, not an extension of PPO

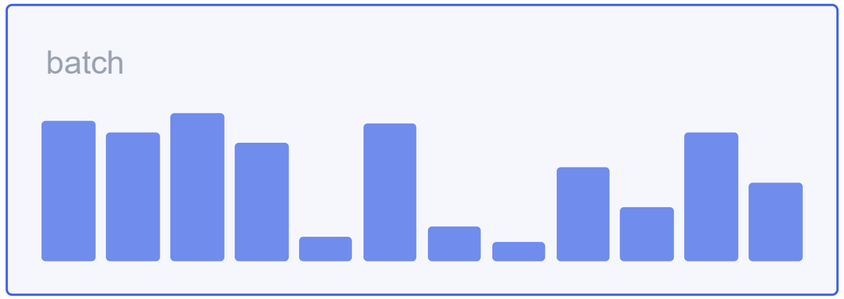
Patches

REINFORCE++: normalize across the batch, not the prompt

Prompt-wise normalization makes the scale of an update depend on which responses happened to be sampled together, can we estimate relative performance on a shared scale across the entire training batch?

Local: each prompt normalized on its own

Global: one normalizer over the whole batch



REINFORCE
no baseline

RLOO, GRPO
a prompt-local baseline

REINFORCE++
global advantage normalization

Patches

DAPO changes four different pieces

1

Clip-Higher

trust-region geometry

2

Dynamic sampling

data selection

3

Token-level loss

gradient aggregation

4

Overlong shaping

reward semantics

Four failure modes: entropy collapse, zero-signal groups, long-CoT weighting, truncation noise

Claim 1: we should clip-higher on the positive side $clip\left(\rho_t, 1-\varepsilon_{low}, 1+\varepsilon_{high}\right)\cdot \hat{A}^{\pi^{\theta_t}}(a_t, s_t)$

a rare exploratory token

old probability 0.01. All the headroom the clip allows is two thousandths.

$0.01(1.2) = 0.012$

a token the model is sure of

old probability 0.9. A nominal upper bound above 1, which is irrelevant.

$0.9 \rightarrow 1.08$

Claim 2: groups with no variance teach nothing

all correct: $R_1 = \cdots = R_G = 1$

Mixed: only group with signal

Patches

A long trace should not count as one vote

GRPO, sample level

$$\frac{1}{G}\sum_{i}\frac{1}{|y_i|}\sum_{t}L_{it}$$

›

DAPO, token level

$$\frac{1}{\sum_{i}|y_i|}\sum_{i}\sum_{t}L_{it}$$

Under token-level aggregation, every token has equal weight regardless of the length of the trace it sits in

Patches

GSPO: match the ratio to the reward

If the verifier assigns one reward to the entire generated response, why are we importance-weighting and clipping every token independently?

a ratio per token

$$\rho_{it} = \frac{\pi_\theta(y_{it}|x, y_{i,<t})}{\pi_{old}(y_{it}|x, y_{i,<t})}$$

›

one ratio per sequence

$$\rho_{i\,seq} = \left(\frac{\pi_\theta(y_i|x)}{\pi_{old}(y_i|x)} \right)^{1/|y_i|}$$

An exponentiated average log-ratio. GSPO reports that this improves stability, notably for MoE training, where tiny token-probability inconsistencies make token ratios especially noisy

Patches

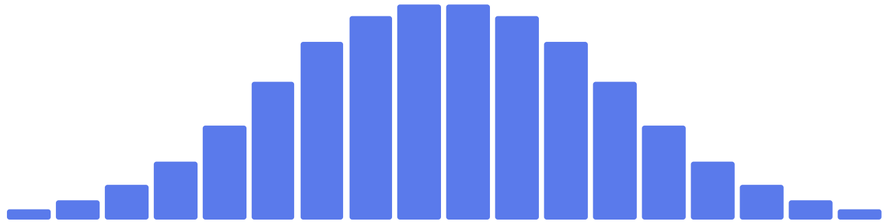
SAPO: no cliff

PPO-style clipping abruptly caps the gradient once the importance ratio crosses a fixed boundary, which can discard useful learning signal from otherwise informative tokens and trajectories…

hard clip: a step function

far below $\rho = 1$ far above

SAPO: a smooth temperature gate



far below $\rho = 1$ far above

$$g_t \sim w_\tau(\rho_t) A_t \nabla \log \pi_t$$

TRPO	PPO	SAPO
a hard KL constraint	hard ratio clipping	a continuous soft trust region

Patches

Two opposite token pathologies, one root

ASPO

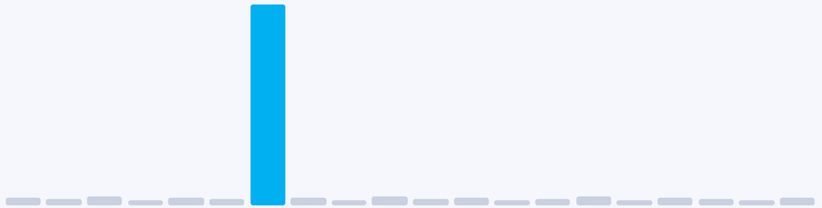
Rich get richer

Because the advantage is shared, the ratio stops being only a distribution correction. It becomes a relative gradient weight among tokens.

ASPO reverses the positive-sample ratio weighting, so lagging positive tokens get more and already-reinforced tokens get less.

STAPO

Accidental tokens dominate



gradient magnitude per token ≈ 0.01% of tokens

Very low probability, low local entropy, sitting inside a positive-reward trajectory, with little causal relation to its success.

Because $A_i \nabla \log \pi(y_t)$ applies to every token, those tokens inherit the full positive outcome. S2T detects and silences them, then renormalizes.

Both exist because the sequence reward is copied indiscriminately onto tokens

Axes

The genealogy

REINFORCE

Monte-Carlo variance, poor credit

baseline, actor-critic, GAE

the policy can move destructively far

TRPO

the constrained solve is expensive

PPO

VAPO fix the critic

A*-PO skip the estimate

RLOO, GRPO

bad normalization › Dr. GRPO

prompt-local normalization › REINFORCE++

entropy collapse, dead groups › DAPO

the hard clip kills gradient › CISPO

wrong ratio granularity › GSPO

the clip boundary is binary › SAPO

IS overweights confident tokens › ASPO

accidental tokens dominate › STAPO

sequence reward is too coarse › GiGPO

Axes

Three questions, not twenty acronyms

A What should A_t mean?

REINFORCE	$A_t \approx R$
PPO	$A_t \approx \mathrm{GAE}(V_\varphi)$
GRPO	$A_t \approx \frac{R_i - R_x}{\sigma_x}$
GiGPO	$A_t = A_{ep} + \omega A_{step}$
A*-PO	$A_t \to A_\star$

credit assignment, variance, computation

B What happens to $\nabla \log \pi_t$ as the policy moves?

TRPO	$D_{KL} < \delta$
PPO	ρhard-clipped
DAPO	$\varepsilon_+ \uparrow$
CISPO	ρ capped, gradient retained
GSPO	$\rho_t \to \rho_{seq}$
SAPO	hard clip › smooth gate
ASPO	$\rho_+ \to$ reversed

trust region, importance weighting, clipping

C Which tokens should receive the gradient?

REINFORCE	all of them
GRPO	all of them
STAPO	some rare tokens are harmful
ASPO	not the same directional dynamics
GiGPO	steps need local credit
DAPO	long traces must not vanish

token and step-selective credit, without paying for a critic